

Sécurité sous Linux

- Introduction
- ACLs: Liste de contrôle d'accès
- Droits spéciaux: setuid, setgid, sticky bit
- SELinux

Gestion des Droits d'accès sous Linux

Autorisations traditionnelles sous Linux: rappel

- Les systèmes Linux utilisent un modèle de **contrôle d'accès** basé sur des **autorisations de fichiers traditionnelles**.
- Chaque fichier ou répertoire est associé à **un propriétaire** et à **un groupe**, auxquels sont attribuées des autorisations spécifiques, représentées par les trois catégories : **lecture** (r), **écriture** (w), et **exécution** (x)
 - Utilisateur propriétaire (u)
 - Groupe propriétaire (g)
 - Autres utilisateurs (o)
 - Lecture (r)
 - Écriture (w)
 - Exécution (x)

```
[bsanae@192 ~]$ ls -l Fichier.txt
-rw-rw-r--. 1 bsanae bsanae 22 Dec 20 16:53 Fichier.txt
[bsanae@192 ~]$
```

Gestion des Droits d'accès sous Linux

Autorisations traditionnelles sous Linux: limitations

- Seul le **propriétaire** du fichier et les membres du **groupe** associé ont des **autorisations spécifiques**.
- Les **autres utilisateurs** sont limités à des **autorisations génériques**, telles que la lecture seule.
- **Difficulté** à accorder des autorisations spécifiques à un utilisateur **sans le rendre propriétaire du fichier**.
- Les changements d'autorisation s'appliquent **de manière uniforme** à tous les membres du groupe
- Partager un fichier avec **un utilisateur spécifique** tout en maintenant les restrictions pour d'autres utilisateurs peut être **compliqué**.
- **Les autorisations s'appliquent uniformément** à tous les fichiers et sous-répertoires dans un répertoire, ce qui peut être trop large dans certaines situations.

Gestion des Droits d'accès sous Linux

Concept des ACLs

- Une **ACL**, ou **Access Control List** (**liste de contrôle d'accès**) est une **liste de permissions** sur un fichier, un répertoire ou une arborescence, **ajoutée aux permissions « classiques »** de ce fichier.
- Ces permissions concernent des **utilisateurs et/ou des groupes** définis.
- Au moyen des ACL, **on peut étendre le nombre d'utilisateurs et de groupes** ayant **des droits sur un même fichier**.
- Avec les **ACL**, on peut (entre autres) **ajouter à un fichier d'autres utilisateurs et groupes** et **définir leurs droits séparément**.
- Les **ACL** sont très utiles (voire **indispensables**) dans des environnements informatiques axés sur **le travail collaboratif et mutualisé**.

Gestion des Droits d'accès sous Linux

Avant les ACLs

- Dans certains cas n'affecter des droits que pour **3 catégories** d'utilisateurs (soi, son groupe et les autres) représente une **limitation importante**.
- Par exemple, on peut vouloir mettre **son devoir en lecture et écriture pour son binôme**, mais **pas pour le reste de la promo**.
- On atteint ce résultat avec **le système de permissions classiques** en **stockant le fichier de son devoir dans un répertoire façade** avec les droits suivants :

```
└─ [drwx--x--x] Binome
   └─ [-rw-rw-r--] devoir_983.tex
```

- Le dossier **Binome** est **accessible à tous**, mais **il est impossible d'en lister son contenu** (pas de droit de lecture). Il suffit alors de **communiquer à son binôme le nom du fichier partagé pour qu'il puisse le modifier**.

Gestion des Droits d'accès sous Linux

Vérifier que le noyau supporte les ACLs

- La commande suivante permet de vérifier **si le noyau prend en charge les ACL**. Elle doit retourner '**y**' pour le type système de fichier qui installé dans votre partition système.

```
[root@server1 emsi]# grep -i acl /boot/config*  
CONFIG_EXT4_FS_POSIX_ACL=y  
CONFIG_XFS_POSIX_ACL=y  
CONFIG_FS_POSIX_ACL=y  
CONFIG_TMPFS_POSIX_ACL=y  
CONFIG_EROFS_FS_POSIX_ACL=y  
CONFIG_NFS_V3_ACL=y  
CONFIG_NFSD_V3_ACL=y  
CONFIG_NFS_ACL_SUPPORT=m  
CONFIG_CEPH_FS_POSIX_ACL=y  
[root@server1 emsi]#
```

- Les **ACLs** Linux sont **supportées nativement** sur les distributions basées Red Hat.

```
[bsanae@localhost ~]$ yum list installed | grep acl  
acl.x86_64 2.2.53-1.el8  
@rhel-8-for-x86_64-baseos-rpms  
libacl.x86_64 2.2.53-1.el8  
@rhel-8-for-x86_64-baseos-rpms  
[bsanae@localhost ~]$
```

Gestion des Droits d'accès sous Linux

ACLs: Commandes

- Il existe deux commandes essentielles : l'une pour **manipuler l'ACL d'un fichier** (**setfacl**) et l'autre pour **la consulter** (**getfacl**).
- Les commandes traditionnelles **chmod** (et **chown**) **ne peuvent accéder aux ACL**

Gestion des Droits d'accès sous Linux

ACLs: Commandes

Commande setfacl: Modifier les droits

- La commande **setfacl** (*set file's ACL*) permet de **définir ou de modifier les ACL** pour un fichier ou un répertoire spécifique.
- Elle offre une **syntaxe flexible** pour **ajouter, modifier ou supprimer** des entrées d'ACL.
- **Exemple:**

```
# setfacl -m u:karim:rw devoir.txt
```

- Cette commande **modifiera** (**-m**) l'ACL de **devoir.txt** en attribuant à l'utilisateur (**préfixe u:**) **karim** les droits **rw** et en lui **refusant le droit d'exécution** (qui n'a pas été mentionné dans la commande).

Gestion des Droits d'accès sous Linux

ACLs: Commandes

Commande setfacl: Modifier les droits

- Les principaux paramètres à connaître sont :

Préfixes :

u: droits pour un **utilisateur**, nommé ou désigné par son uid

g: droits pour un **groupe**, nommé ou désigné par son gid

o: droits pour **other**, le reste du monde

Permissions :

⇒ Elles sont codées dans l'ordre r, w et x.

⇒ On les remplace par - pour une interdiction explicite.

⇒ Ne pas mentionner un droit revient aussi à une interdiction

```
# setfacl -m u:karim:-w- devoir.txt
```

=

```
# setfacl -m u:karim:w devoir.txt
```

Gestion des Droits d'accès sous Linux

ACLs: Commandes

Commande setfacl: modifier les droits

- On peut construire des commandes plus complexes en enchaînant les entrées dans l'ACL :

```
# setfacl -m u:karim:rw,g:etudiants:r,o:--- devoir.txt
```

- Cette commande définit des permissions dans l'**ACL** de **devoir.txt** pour l'utilisateur **karim**, le groupe **etudiants** et le reste du monde.
- La commande fonctionne bien sûr aussi de **manière récursive** (**option -R**) :

```
# setfacl -Rm u:karim:rw Devoirs
```

- Cette commande **modifie l'ACL** de **tous les fichiers situés sous Devoirs/** en attribuant une permission de **lecture et d'écriture** à l'utilisateur **karim**.

Gestion des Droits d'accès sous Linux

ACLs: Commandes

Commande setfacl: Retirer les droits

- L'option **-b** de la commande **setfacl** est utilisée pour **supprimer toutes les ACL d'un fichier ou d'un répertoire**. Cela signifie qu'elle réinitialise complètement les autorisations, en supprimant toutes les entrées d'ACL existantes.

```
$ setfacl -b fichier.txt
```

- Cette commande **supprime toutes les ACL associées au fichier fichier.txt**. Les autorisations par défaut basculent vers les permissions de fichier standard (**propriétaire, groupe, autres**).

```
$ setfacl -x u:karim,g:etudiants fichier.txt
```

- Cette commande **retire les permissions propres** à **karim** et au **groupe étudiants**.

Exercices 1 et 2, atelier 1

TP11

Commande getfacl:

- Cette commande suivie d'un nom de fichier affiche l'ACL de ce fichier (**get file's ACL** « récupérer l'ACL du fichier »).

Exercice 3, atelier 1 TP11

Gestion des Droits d'accès sous Linux

ACLs par défaut

- Les **ACLs par défaut** permettent de **donner des permissions ACL en héritage pour tout sous-répertoire ou fichier créé dans un répertoire.**
- Toutefois, ces **ACLs par défaut** ne **s'appliquent pas aux objets déjà présents dans le répertoire**
- **Exemple:** pour empêcher tous **les autres** en termes de permissions pour tout nouveau fichier ou sous-répertoire créé :

```
$ setfacl -m d:o::- repertoire
```

- Les permissions ACL par défaut d'un répertoire (**d:**) s'annulent par **setfacl -k.**

Exercice 4, atelier 1 TP11

Gestion des Droits d'accès sous Linux

Masque dans les ACLs

- **Le masque** est une permission qui **limite les autorisations effectives** pour les groupes et entrées spécifiques.
- Exemple d'une ACL avec masque (**getfacl**):

```
[emsi@server1 ~]$ getfacl document.txt
# file: document.txt
# owner: emsi
# group: emsi
user::rw-
user:user1:rw-          #effective:r--
group::r--
group:group1:rwx        #effective:r--
mask::r--
other::r--
```

- Définir un masque :

```
$ setfacl -m m::rwx fichier
```

Exercice 5, atelier 1 TP11

Gestion des Droits d'accès sous Linux

Droits spéciaux: Introduction

- **Les droits spéciaux** sous Linux sont des **droits supplémentaires** qui peuvent être accordés à un fichier ou à un répertoire.
- Ils sont utilisés pour **modifier le comportement normal** des fichiers et des répertoires.
- Il existe trois droits spéciaux :
 - ⇒ **Setuid**: Ce droit permet à **un utilisateur non root** d'exécuter un programme avec les droits du **propriétaire du programme**.
 - ⇒ **Setgid**: Ce droit permet à **un utilisateur non root** d'effectuer des opérations sur un répertoire avec **les droits du groupe propriétaire du répertoire**.
 - ⇒ **Sticky bit**: Ce droit empêche **les utilisateurs non root** de **supprimer ou de modifier** les fichiers d'un répertoire

Gestion des Droits d'accès sous Linux

Le droit setuid

- Le droit **setuid** est utilisé pour **exécuter des programmes avec les droits du propriétaire du programme**.
- Cela peut être utile pour **exécuter des programmes qui nécessitent des privilèges élevés**, tels que des programmes d'administration système.
- Cette fois le **s** est présent à **la place du bit exécutable sur le secteur user**.
- Si le **droit x n'est pas accordé** à la **catégorie user**, à la place de **s** c'est la lettre **S** qui apparaîtra.
- **Par exemple**: le programme **sudo** utilise le droit **setuid** pour **permettre aux utilisateurs non root** d'exécuter des commandes avec les droits du **super-utilisateur**.

```
[bsanae@192 ~]$ ls -l /usr/bin/sudo
---s--x--x. 1 root root 165536 Jan 16  2023 /usr/bin/sudo
[bsanae@192 ~]$
```

Gestion des Droits d'accès sous Linux

Le droit setuid

setuid: Activation du droit

- Pour **activer** le droit **setuid** en mode symbolique:

```
$ chmod u+s mon_programme
```

- Vous pouvez utiliser la commande chmod suivie du code de permission **4xxx**, où xxx représente les bits de permission pour le fichier.

```
$ chmod 4xxx mon_programme
```

- Désactiver** le droit **setuid**:

```
$ chmod u-s mon_programme
```

ou

```
$ chmod 0xxx mon_programme
```

Exercice 1, atelier 2 TP11

Gestion des Droits d'accès sous Linux

Le droit setgid

- Sur les systèmes UNIX, **Setuid** n'a pas d'effet sur les répertoires. L'attribut **setuid** est **principalement conçu pour les fichiers exécutables**.
- Contrairement au bit **setuid**, le bit **setgid** a un effet **à la fois sur les fichiers et les répertoires**.
- le fichier qui a le bit **setgid** activé, lorsqu'il est exécuté, **au lieu de s'exécuter avec les privilèges du groupe de l'utilisateur** qui l'a démarré, **s'exécute avec les privilèges du groupe propriétaire du fichier**.
- Lorsqu'il est utilisé **sur un répertoire**, le bit **setgid** modifie le comportement standard de sorte que **le groupe de fichiers créés dans ledit répertoire ne sera pas celui de l'utilisateur qui les a créés, mais celui du parent**.
- Ceci est souvent utilisé pour faciliter le partage de fichiers (**les fichiers seront modifiables par tous les utilisateurs faisant partie dudit groupe**).

Gestion des Droits d'accès sous Linux

Le droit setgid sur les fichiers

Effet sur les fichiers exécutables :

- Lorsque **setgid** est activé sur un fichier exécutable, le programme s'exécute **avec les privilèges du groupe propriétaire du fichier**, au lieu du groupe de l'utilisateur qui lance le programme.
- **Exemple:**
 - Supposons que l'utilisateur **john** appartient au groupe **users** mais que le fichier **program** appartient au groupe **admin**.
 - Avec **setgid**, si **john** exécute le fichier **program**, il exécute le fichier avec les privilèges du groupe **admin**, et non du groupe **users**.

Gestion des Droits d'accès sous Linux

Le droit setgid sur les répertoires

Effet sur les répertoires:

- Lorsque le **setgid** est appliqué à un répertoire, tous les fichiers créés à l'intérieur de ce répertoire **héritent du groupe du répertoire**, plutôt que du groupe principal de l'utilisateur qui crée le fichier.
- **Exemple:**
 - Supposons que le répertoire **/home/shared** appartienne au groupe **shared** et qu'un utilisateur **alice** du groupe **staff** crée un fichier dans ce répertoire.
 - Même si **alice** est dans le groupe **staff**, le fichier créé aura le groupe **shared**, car il hérite du groupe du répertoire grâce à **setgid**.

Gestion des Droits d'accès sous Linux

Le droit setgid

- Tout comme le **setuid**, le bit **setgid** peut facilement être repéré (dans ce cas sur un répertoire **/projet**) :

```
[root@localhost bsanae]# ls -ld /projet
drwxrwsr-x. 2 root collaborateurs 6 Dec 26 17:46 /projet
```

- Cette fois le **s** est présent à la place du bit exécutable sur le secteur groupe.
- Si le **droit x n'est pas accordé** à la **catégorie group**, à la place de **s** c'est la lettre **S** qui apparaîtra.

setgid: activation du droit:

{
chmod **2xxx** /repertoire
OU
chmod **g+s** /repertoire

setgid: désactivation du droit:

{
chmod **0xxx** /repertoire
OU
chmod **g-s** /repertoire

Exercice 2, atelier 2 TP11

Gestion des Droits d'accès sous Linux

Le Sticky Bit

- Ce droit spécial, traduit en "**bit collant**", a surtout un rôle important sur les répertoires.
- Il régleme le **droit w** sur le répertoire, en **interdisant à un utilisateur quelconque de supprimer un fichier dont il n'est pas le propriétaire**.
- Ce droit **noté symboliquement t** occupe par convention la place du **droit x** sur la catégorie **other** de ce répertoire, mais bien entendu il **ne supprime pas le droit d'accès x (s'il est accordé)**.
- Si le **droit x n'est pas accordé** à la **catégorie other**, à la place de **t** c'est la lettre **T** qui apparaîtra.
- Pour positionner ce droit :

```
# chmod 1xxx /repertoire
```

ou

```
# chmod +t /repertoire
```

- Pour retirer ce droit:

```
# chmod 0xxx /repertoire
```

ou

```
# chmod -t /repertoire
```

Exercice 3, atelier 2 TP11

TP n° 12 : Security Enhanced Linux (SELinux)

Atelier 1 : Simulation d'une attaque via un serveur web compromis

**Comment rendre impossible l'accès aux autres
répertoires à partir de service web apache
(httpd) ?**

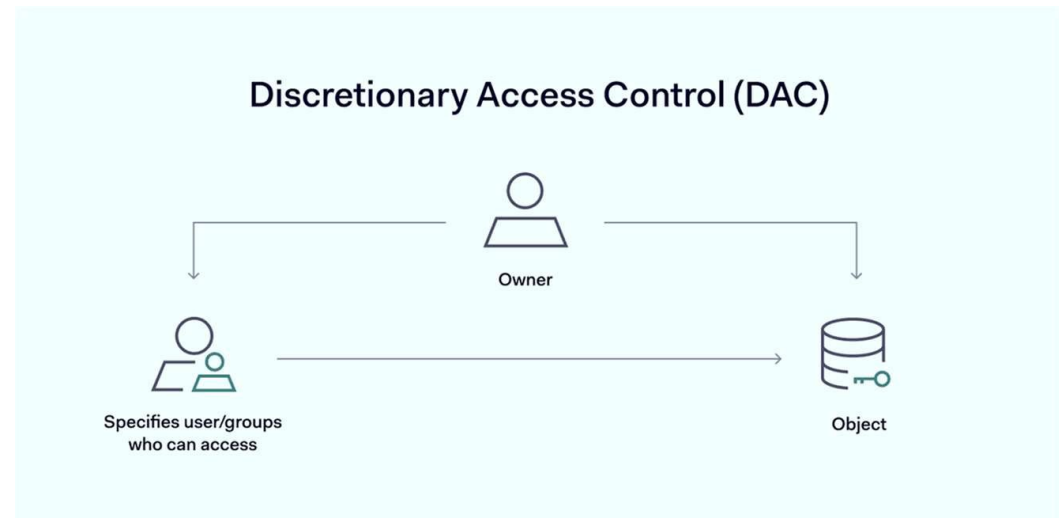
Sécurité sous Linux: SELinux

SELinux: Introduction

DAC: Discretionary Access Control

Les droits d'accès **discretionnaires** forment un modèle de contrôle d'accès qui permet au **propriétaire d'une ressource**, telle qu'un fichier, un dossier, un processus ou un périphérique, **d'accorder ou de refuser** l'accès à d'autres utilisateurs ou groupes.

- ⇒ Les fichiers et processus ont des **propriétaires**.
- ⇒ Le propriétaire d'un fichier peut être un **utilisateur, un groupe, ou autre**
- ⇒ Les utilisateurs ont la possibilité de **modifier les autorisations d'accès à leurs propres fichiers**.
- ⇒ L'utilisateur root dispose d'un **contrôle d'accès total**.



Exemples: Droits ordinaires (r,w,x), Droits spéciaux (setuid, setgid, stiky bit), les ACLs

Sécurité sous Linux: SELinux

SELinux: Introduction

Exemple pratique : Apache compromis

- **Scénario :**
 - ⇒ Apache est configuré pour servir des fichiers dans /var/www/html.
 - ⇒ **Les permissions ordinaires et ACLs** permettent à Apache de lire et écrire dans ce répertoire.
 - ⇒ Une faille dans Apache permet à un attaquant d'exécuter du code malveillant avec les privilèges d'Apache.
- **Impact :**
 - ⇒ L'attaquant peut accéder à tous les fichiers dans /var/www/html, y compris des fichiers contenant des mots de passe ou des informations sensibles.
 - ⇒ Il peut écrire des fichiers malveillants dans le répertoire pour exécuter des attaques ultérieures (ex. : un script PHP malveillant).
 - ⇒ Si Apache a accès à des fichiers système ou à des ports réseau critiques, ces ressources peuvent être compromises.

Sécurité sous Linux: SELinux

SELinux: Introduction

- **SELinux** (**Security-Enhanced Linux**) est une architecture de sécurité pour systèmes Linux® qui permet aux administrateurs de mieux contrôler les accès au système. C'est un modèle de sécurité que l'on peut ajouter au système standard de Linux afin **d'augmenter la sécurité** face aux diverses attaques que subit le système.
- **SELinux** fournit **une couche supplémentaire** de sécurité du système ⇒ il répond fondamentalement à la question suivante :

Le **<sujet>** peut-il faire **<action>** au **<objet>** ?

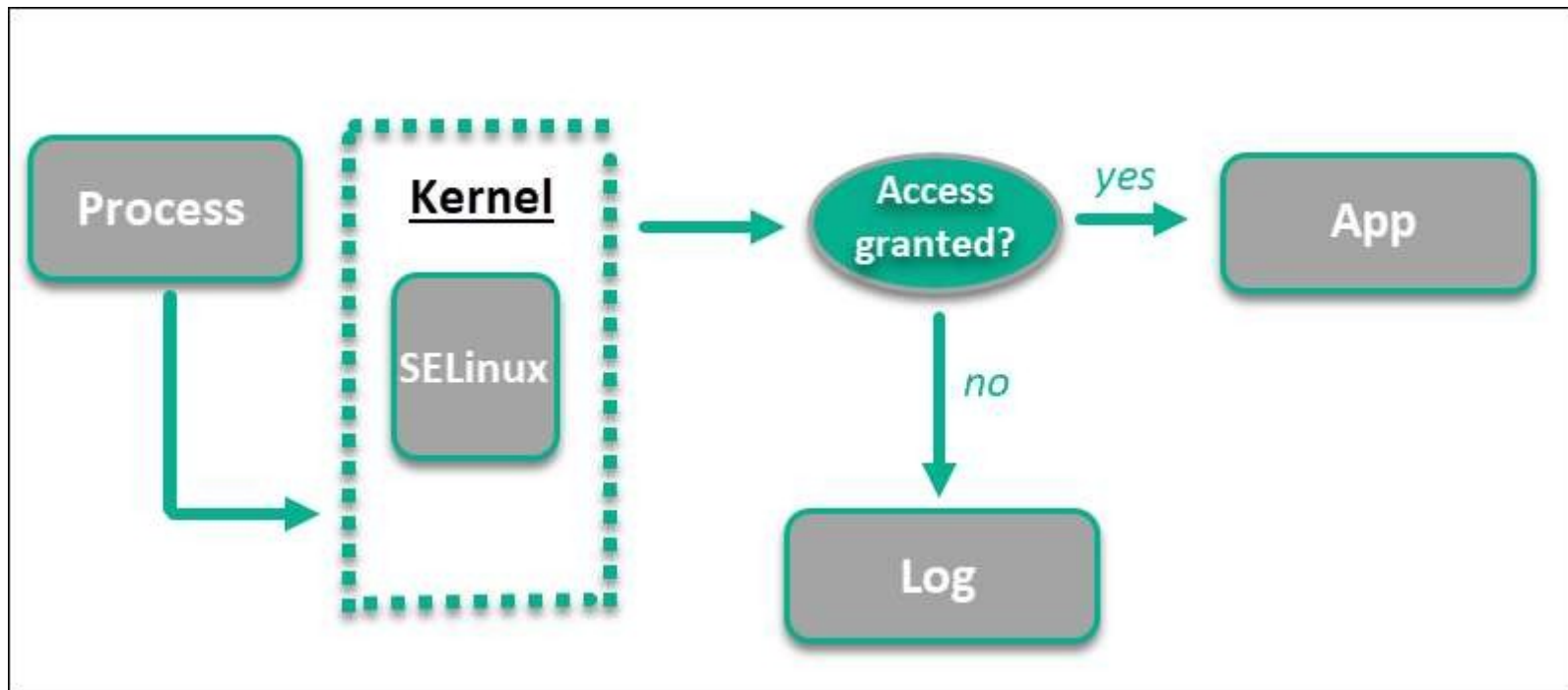
- Exemple:

Un site web peut-il **accéder** aux **fichiers** des répertoires personnels des utilisateurs ?

Sécurité sous Linux: SELinux

SELinux: principe de base

- SELinux est **une couche supplémentaire de sécurité** du système.
- L'objectif principal de SELinux est de **protéger les données des utilisateurs** contre les services du système qui ont été **compromis** (attaqués) .



Sécurité sous Linux: SELinux

SELinux: Introduction

MAC: Mandatory Access Control

- Le **contrôle d'accès mandataire** définit une politique de sécurité pour le système à laquelle même root doit se soumettre.
- L'implémentation la plus connue du MAC sous Linux est **SELinux**
- Cette implémentation supporte le modèle de sécurité **TE (Type Enforcement)**.
- Le TE consiste à **labelliser le système**: Chaque entité du système se voit attribuer **un label**.
- Ensuite, il faut fournir **une politique de sécurité** définissant **les actions autorisées** entre les labels.
- Ce qui n'est pas **autorisé explicitement** par la politique est **interdit**.
- **Entité** = fichiers, répertoires, processus, utilisateurs, périphériques, ports

- **Les règles de politique SELinux sont vérifiées après les règles DAC.**
 - ⇒ **DAC:** Contrôle d'accès discrétionnaire (autorisations de base).
 - ⇒ **SELinux:** Contrôle d'accès obligatoire (restrictions supplémentaires).
 - ⇒ **Ordre des vérifications:** DAC est vérifié en premier, puis SELinux si nécessaire.
 - ⇒ **Aucun enregistrement SELinux si DAC refuse l'accès:** Si vous n'avez pas l'autorisation de base, SELinux n'intervient pas et aucun message d'erreur spécifique à SELinux n'est enregistré.

Forces des DAC:

- **Flexibilité:** Les utilisateurs peuvent partager des données et collaborer facilement avec d'autres sans nécessiter d'intervention administrative pour chaque modification de politique.
- **Facilité de manipulation:** La modification des politiques est un processus très simple.

Limites des DAC:

- **Risques de sécurité:** Étant donné que les utilisateurs contrôlent leurs propres autorisations, des erreurs de configuration peuvent violer le principe du moindre privilège, entraînant un accès non autorisé.

Sécurité sous Linux: SELinux

DAC vs MAC

Forces des MAC:

- **Sécurité stricte:** le MAC fournit un environnement très sécurisé en appliquant strictement les règles d'accès.
- **Confidentialité:** Garantit que les informations sensibles ne sont accessibles qu'aux utilisateurs disposant de l'autorisation appropriée.
- **Contrôle centralisé:** Les administrateurs ont un contrôle total et direct sur les politiques d'accès, réduisant ainsi le risque d'accès non autorisé.

Limites des MAC:

- **Complexité:** La gestion et le maintien de politiques d'accès strictes peuvent être complexes et chronophages, nécessitant une intervention directe d'un administrateur système pour la configuration et la gestion de chaque politique
- **Expérience utilisateur:** Étant donné que les utilisateurs n'ont pas la possibilité de modifier les autorisations, cette approche ne convient pas aux environnements dynamiques où le partage de données et la collaboration sont importants.

Sécurité sous Linux: SELinux

Exemple d'un serveur web (3)

Que peut faire SELinux?

SELinux: dire à l'utilisateur s'il peut avoir accès à un fichier

- **Par exemple**, vous pouvez accéder au répertoire `/var/www/html`, mais je ne vais pas vous donner accès à `/var/www/cgi-bin`

```
[student@servera www]$ ls -l
total 0
drwxr-xr-x. 2 root root 6 Mar 21 2022 cgi-bin
drwxr-xr-x. 2 root root 6 Mar 21 2022 html
```

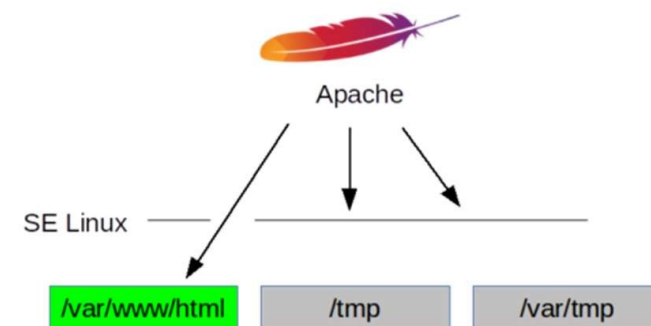
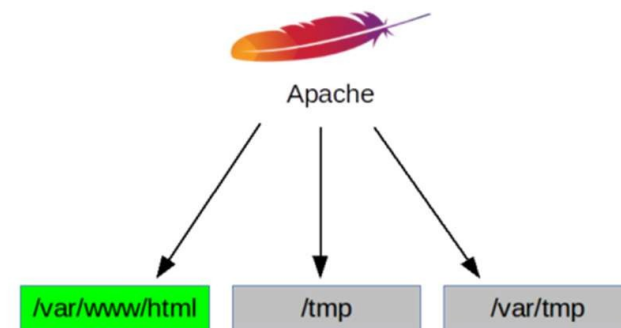
⇒ `/var/www/html` : Contient des fichiers visibles et accessibles publiquement, comme des pages web statiques.

⇒ `/var/www/cgi-bin` : Contient des scripts ou programmes exécutables, souvent plus sensibles.

Sécurité sous Linux: SELinux

Exemple d'un serveur web

- **SELinux** empêcherait donc un utilisateur compromis d'exploiter le processus HTTP pour **accéder à des fichiers ou à des répertoires**, à moins qu'une règle SELinux spécifique ne soit configurée pour permettre un tel accès.
- **SELinux** peut également restreindre l'accès de cet utilisateur à certains fichiers ou répertoires, même s'ils appartiennent au même utilisateur ou au même groupe.
- Ainsi, grâce à **SELinux**, un même utilisateur peut être **autorisé à accéder** à un répertoire mais pas à un autre, ou à un fichier mais pas à un autre, en fonction des **règles de sécurité définies**.



Sécurité sous Linux: SELinux

SELinux: Concepts utilisés

- **Sujet (Subject): l'entité qui effectue une action**, inclut:
 - ⇒ **Processus** : Ex: un serveur web (Apache) ou un démon système.
 - ⇒ **Service** : Un service en cours d'exécution, comme ssh ou cron.
 - ⇒ **Utilisateur** : utilisateur système ou ordinaire.
- **Objet (Object): sur lequel une action est réalisée par le sujet**. Inclut :
 - ⇒ **Fichier** : Les fichiers du système, configuration, journaux ou exécutables.
 - ⇒ **Répertoire** : Les dossiers qui contiennent des fichiers.
 - ⇒ **Socket** : Les interfaces réseau utilisées par les services.
 - ⇒ **Port** : Les points d'accès réseau, Ex: 80 pour HTTP ou 22 pour SSH.
- **Règles (Policy): ensembles de règles qui contrôlent quelles actions un sujet peut effectuer sur un objet**. Ces règles sont stockées dans la base de données de SELinux et déterminent si une action est permise ou refusée.
 - ⇒ **Permettre (allow)** : L'action demandée par le sujet est autorisée.
 - ⇒ **Refuser (deny)** : L'action est bloquée

Sécurité sous Linux: SELinux

SELinux: Labeling = étiquetage

- **Les politiques SELinux** sont des règles de sécurité qui définissent la manière dont des processus spécifiques accèdent aux fichiers, répertoires et ports pertinents.
- Chaque ressource, comme un fichier, un processus, un répertoire ou un port, possède une étiquette appelée **contexte SELinux**.
- Les étiquettes SELinux ont **les champs user, role, type et security level**.

<i>SELinux User</i>	<i>Role</i>	<i>Type</i>	<i>Level</i>	<i>File</i>
unconfined_u	object_r	httpd_sys_content_t	s0	/var/www/html/file2

- Dans RHEL, les règles sont définies par défaut en utilisant **le contexte type**:
 - ⇒ Les processus ayant **le type httpd_t** (serveur web) ne peuvent lire que des fichiers ayant des types spécifiques comme **httpd_sys_content_t**.
 - ⇒ Un processus ayant le type **user_t** (utilisateur normal) ne pourra pas accéder aux fichiers ayant le type **shadow_t** (fichier des mots de passe).

Sécurité sous Linux: SELinux

Label SELinux

Exemple 1:

SELinux User /	Role /	Type /	Level /	File /
unconfined_u	object_r	httpd_sys_content_t	s0	/var/www/html/file2

- **unconfined_u**: C'est l'utilisateur SELinux associé à cet objet (ici le fichier file2).
 - ⇒ Dans ce cas, l'utilisateur est unconfined_u, ce qui signifie qu'il n'est pas soumis à des politiques strictes de confinement SELinux.
 - ⇒ Par défaut, pour des services comme Apache, le fichier peut être associé à des utilisateurs tels que system_u.
- **object_r**: Le rôle associé à cet objet.
 - ⇒ Ici, indique que ce contexte est celui d'un objet, comme un fichier ou un répertoire.
- **httpd_sys_content_t**: Le type SELinux est le plus important car il détermine ce que le processus peut faire avec l'objet.
 - ⇒ Ici, le type est httpd_sys_content_t, qui est spécifiquement utilisé pour les fichiers et répertoires accessibles en lecture par le serveur Apache (httpd). Apache peut lire et servir des fichiers marqués comme **httpd_sys_content_t**.
 - ⇒ Apache ne peut pas écrire ou exécuter ces fichiers, à moins qu'ils soient marqués avec un autre type (comme **httpd_sys_rw_content_t** pour lecture/écriture).
- **Level (s0)**: Le niveau de sensibilité ou de classification du fichier.

Sécurité sous Linux: SELinux

Label SELinux

Exemple 2:

```
system_u:object_r:var_log_t:s0 /var/log/messages
```

- **system_u**: L'utilisateur SELinux qui possède cet objet
⇒ Ici, cela signifie que le fichier est un objet appartenant au système.
- **object_r**: Le rôle attribué à cet objet (ici fichier ou répertoire)
- **var_log_t**: Le type SELinux de cet objet.
⇒ Ici, cela désigne un fichier utilisé pour les journaux système (/var/log).
- **s0**: Le niveau de sensibilité de cet objet. Ici, s0 est le niveau par défaut.
- **/var/log/messages**: Le fichier en question.

Prenons l'exemple où un processus **Apache (httpd)** veut accéder au fichier **/var/log/messages** :

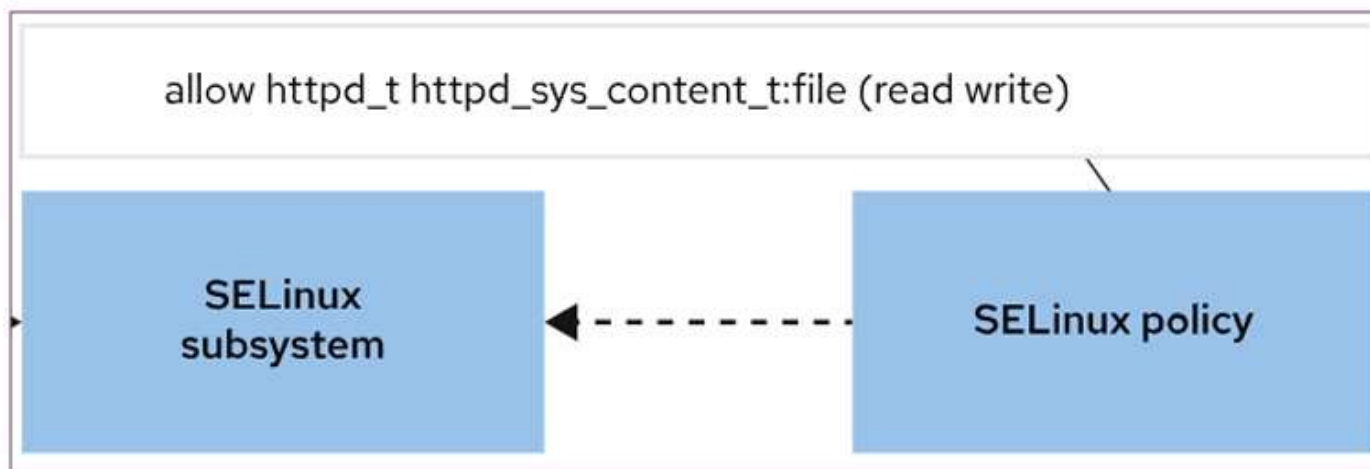
- ⇒ Si SELinux est activé, il vérifiera si le type associé à Apache (par exemple, **httpd_t**) a la permission d'accéder au type **var_log_t**.
- ⇒ Si aucune règle explicite ne permet cet accès, SELinux **bloquera** l'action, même si les permissions classiques (comme celles définies par chmod) le permettent.

Sécurité sous Linux: SELinux

Politique SELinux: Labeling

- L'étiquette de contexte correspond à **une règle de politique SELinux** définie pour **autoriser un processus à accéder à la ressource étiquetée**.
- Par défaut, **une politique SELinux n'autorise aucun accès**, sauf si une règle explicite en accorde un.
- Si aucune règle d'autorisation n'est définie, **tous les accès sont interdits**.

Allow resource_type1 access to resource_type2 (access mode)



Sécurité sous Linux: SELinux

Politique SELinux: Labeling

Comment SELinux résoud le problème du serveur Apache (httpd)?

- Il se trouve que **apache** fonctionne dans **un contexte de sécurité** particulier de SELinux, il a le type **httpd_t**.

```
system_u:system_r:httpd_t:s0 15228 ? 00:00:00 httpd
```

- Le fichier **shadow**, possède lui aussi **un contexte de sécurité**, et son type est **shadow_t**.

```
system_u:object_r:shadow_t:s0 /etc/shadow
```

- Avec **SELinux**, pour que **apache** puisse lire ou écrire le fichier **shadow**, il faut une **règle explicite** qui permet aux processus de type **httpd_t** de lire ou écrire dans un fichier de type **shadow_t**. Il faudrait donc une règle comme celle-ci dans les politiques de sécurité de **SELinux**:

```
allow httpd_t shadow_t : file { read write }
```

Sécurité sous Linux: SELinux

Confined vs Unconfined

confined:

- Un processus **confined** (Confiné) est soumis à **des règles strictes de sécurité** de SELinux, avec des accès très limités (politiques SELinux standard).
- Ils ne peuvent effectuer que les actions spécifiquement autorisées, renforçant ainsi la sécurité.
- Ses actions sont limitées par les règles de sécurité, ce qui permet un contrôle strict des permissions et des interactions.

unconfined :

- Un processus **unconfined** (non confiné) **n'est pas restreint par SELinux**, ou du moins, ses actions sont beaucoup moins surveillées. Cela signifie qu'il a des privilèges étendus et n'est pas soumis aux mêmes restrictions qu'un processus confiné.
- Utilisés lorsque la sécurité n'a pas besoin d'être aussi élevée ou que la configuration SELinux spécifique n'est pas encore en place

Sécurité sous Linux: SELinux

Exemples courants de types SELinux

httpd_sys_content_t: Contenu des fichiers accessibles par Apache (HTTPD).

```
[fadili@localhost fadili]$ ls -Z /var/www/html/index.html
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html
[fadili@localhost fadili]$
```

device_t : Fichiers associés aux périphériques.

```
[fadili@localhost fadili]$ ls -Z /dev/tty1
system_u:object_r:tty_device_t:s0 /dev/tty1
[fadili@localhost fadili]$ ls -Z /dev/sda
system_u:object_r:fixed_disk_device_t:s0 /dev/sda
[fadili@localhost fadili]$
```

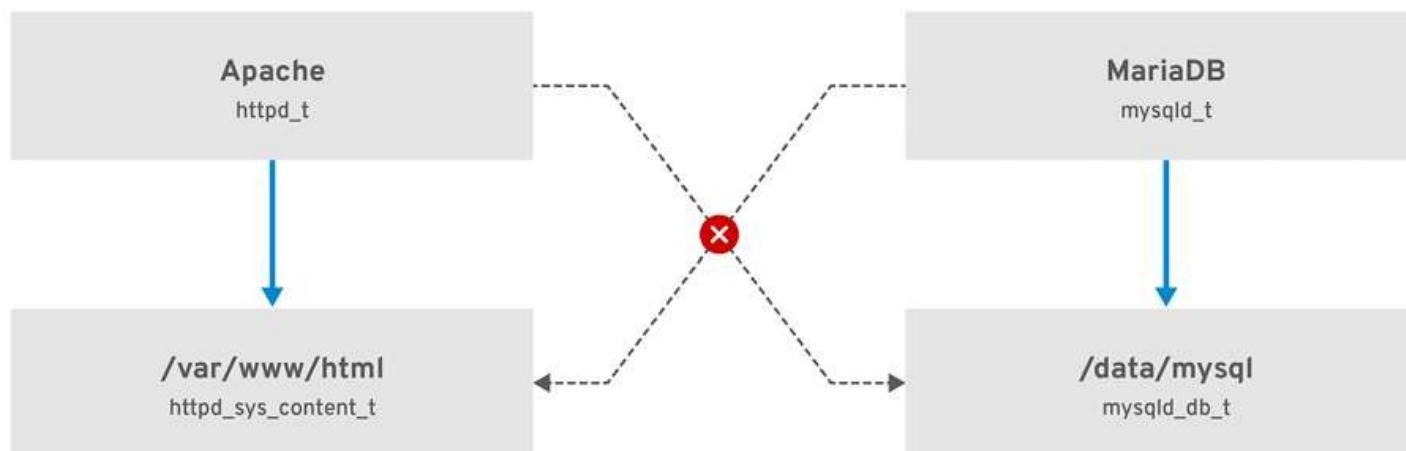
home_dir_t : Répertoires personnels des utilisateurs

```
[fadili@localhost fadili]$ ls -Zd /home/fadili
unconfined_u:object_r:user_home_dir_t:s0 /home/fadili
[fadili@localhost fadili]$ ls -Zd /root
system_u:object_r:admin_home_t:s0 /root
[fadili@localhost fadili]$
```

Sécurité sous Linux: SELinux

Politique SELinux: Exemple

Exemple de la manière dont SELinux peut aider à faire fonctionner Apache et MariaDB de manière sécurisée sur le même serveur



- SELinux permet au processus **Apache** fonctionnant en tant que **httpd_t** d'accéder au répertoire **/var/www/html/** et refuse à ce même processus l'accès au répertoire **/data/mysql/** dont le contexte est **mysqld_db_t**.
- D'autre part, le processus **MariaDB** exécuté en tant que **mysqld_t** est capable d'accéder au répertoire **/data/mysql/** et SELinux refuse également correctement au processus de type **mysqld_t** d'accéder au répertoire **/var/www/html/** étiqueté en tant que **httpd_sys_content_t**.

Sécurité sous Linux: SELinux

SELinux: modèle de sécurité

- Le système d'exploitation **contraint** les utilisateurs ou programmes dans leurs **accès aux ressources**:
 - ⇒ **Les objets** sont des ressources système : fichiers, répertoires, sockets et autres périphériques,
 - ⇒ **Les sujets** sont les process (commandes ou programmes).
 - ⇒ **Sujets et objets** se voient attribué un contexte de sécurité.
 - ⇒ Le système détermine l'accès à un objet par un sujet selon la **politique de sécurité chargée**.
 - ⇒ Le sujet ne peut pas modifier le contexte de sécurité d'un objet.
 - ⇒ Ainsi un sujet **est confiné**.

Sécurité sous Linux: SELinux

Consulter le contexte de sécurité

- Consulter les informations de l'actuel contexte SELinux avec l'argument **-Z** (cet argument est utilisé par **ps**, **ls**,).
- **Exemple 1 : liste des contextes pour tous les processus**

```
max kernel policy version. 33
[root@localhost ~]# ps -ef -Z
LABEL                                UID      PID    PPID  C  STIME TTY      TIME CMD
system_u:system_r:init_t:s0         root      1       0  0  00:21 ?        00:00:16 /usr/lib/systemd/systemd --sw
system_u:system_r:kernel_t:s0       root      2       0  0  00:21 ?        00:00:00 [kthreadd]
system_u:system_r:kernel_t:s0       root      3       2  0  00:21 ?        00:00:00 [rcu_gp]
system_u:system_r:kernel_t:s0       root      4       2  0  00:21 ?        00:00:00 [rcu_par_gp]
system_u:system_r:kernel_t:s0       root      5       2  0  00:21 ?        00:00:00 [slub_flushwq]
system_u:system_r:kernel_t:s0       root      7       2  0  00:21 ?        00:00:00 [kworker/0:0H-events_highpri]
system_u:system_r:kernel_t:s0       root     10       2  0  00:21 ?        00:00:00 [mm_percpu_wq]
system_u:system_r:kernel_t:s0       root     11       2  0  00:21 ?        00:00:00 [rcu_tasks_rude_]
system_u:system_r:kernel_t:s0       root     12       2  0  00:21 ?        00:00:00 [rcu_tasks_trace]
system_u:system_r:kernel_t:s0       root     13       2  0  00:21 ?        00:00:07 [ksoftirqd/0]
system_u:system_r:kernel_t:s0       root     14       2  0  00:21 ?        00:00:08 [rcu_sched]
system_u:system_r:kernel_t:s0       root     15       2  0  00:21 ?        00:00:00 [migration/0]
system_u:system_r:kernel_t:s0       root     16       2  0  00:21 ?        00:00:00 [watchdog/0]
system_u:system_r:kernel_t:s0       root     17       2  0  00:21 ?        00:00:00 [cpuhp/0]
system_u:system_r:kernel_t:s0       root     18       2  0  00:21 ?        00:00:00 [cpuhp/1]
system_u:system_r:kernel_t:s0       root     19       2  0  00:21 ?        00:00:00 [watchdog/1]
```

Sécurité sous Linux: SELinux

Consulter le contexte de sécurité

- Exemple 2 : liste des contextes pour le processus httpd

```
[root@localhost ~]# ps -ef -Z | grep httpd
system_u:system_r:httpd_t:s0 root 15228 1 0 11:51 ? 00:00:01 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 15235 15228 0 11:51 ? 00:00:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 15236 15228 0 11:51 ? 00:00:10 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 15237 15228 0 11:51 ? 00:00:07 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache 15238 15228 0 11:51 ? 00:00:07 /usr/sbin/httpd -DFOREGROUND
```

- Exemple 3: contexte des fichiers du dossier /var/www/

```
[root@localhost ~]# ls -alZ /var/www/
total 4
drwxr-xr-x. 4 root root system_u:object_r:httpd_sys_content_t:s0 33 Nov 6 13:25 .
drwxr-xr-x. 22 root root system_u:object_r:var_t:s0 4096 Nov 6 13:25 ..
drwxr-xr-x. 2 root root system_u:object_r:httpd_sys_script_exec_t:s0 6 Aug 30 16:03 cgi-bin
drwxr-xr-x. 3 root root system_u:object_r:httpd_sys_content_t:s0 41 Jan 1 13:01 html
[root@localhost ~]#
```

Sécurité sous Linux: SELinux

Modes de fonctionnement

- **SELinux** propose **trois modes** différents :
 - ⇒ Dans **le mode strict (Enforcing)**, niveau de sécurité le plus élevé: les accès sont restreints, SELinux va **refuser** tous les accès non autorisés.
 - ⇒ **Le mode permissif (Permissive)** peut être considéré comme un mode de débogage. Les règles **SELinux** sont interrogées, les erreurs d'accès sont enregistrées dans les logs, mais **l'accès ne sera pas bloqué**.
 - ⇒ Lorsque **SELinux** est **désactivé (Disabled)**, l'accès n'est pas restreint, et rien n'est enregistré dans les logs.

Sécurité sous Linux: SELinux

Modes de fonctionnement: commandes

- La commande **getenforce** vous informe sur le mode SELinux sur votre machine :

```
[root@localhost bsanae]# getenforce
Enforcing
[root@localhost bsanae]#
```

- La commande **setenforce** permet de basculer temporairement (**jusqu'au prochain redémarrage**) entre les modes strict (**Enforcing**) et permissif (**Permissive**) :

```
Enforcing
[root@localhost bsanae]# setenforce 0
[root@localhost bsanae]# getenforce
Permissive
[root@localhost bsanae]# setenforce 1
[root@localhost bsanae]# getenforce
Enforcing
[root@localhost bsanae]#
```

Ou bien **setenforce permissive**

Ou bien **setenforce enforcing**

- Le mode **disabled** est une option qui doit être configurée dans le **fichier de configuration de SELinux**.

Sécurité sous Linux: SELinux

Modes de fonctionnement: Fichier

- Le **mode SELinux par défaut** est défini dans le fichier :
/etc/selinux/config:

```
enforcing
[root@localhost bsanae]# cat /etc/selinux/config

# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled - No SELinux policy is loaded.
SELINUX=enforcing
# SELINUXTYPE= can take one of these three values:
#   targeted - Targeted processes are protected,
#   minimum - Modification of targeted policy. Only selected processes are protected.
#   mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

- SELINUX** prendra une des trois valeurs **enforcing, permissive ou disabled**. Quant à SELINUXTYPE, on gardera la valeur par défaut targeted, qui garantit la surveillance des principaux services réseau.

Sécurité sous Linux: SELinux

Modes de fonctionnement

Attention !

- Lorsque **SELinux** est exécuté en **mode permissif** ou s'il est désactivé, les nouveaux fichiers créés ne porteront aucune étiquette. Lorsque SELinux sera réactivé, cela pourra poser des problèmes, et il vous faudra donc réétiqueter l'intégralité de votre système:

```
[root@localhost html]# touch /.autorelabel  
[root@localhost html]# reboot
```

- Pour connaître l'état général (activé ou désactivé, mode courant, politique chargée, etc...:

```
[root@localhost ~]# sestatus  
SELinux status:                enabled  
SELinuxfs mount:              /sys/fs/selinux  
SELinux root directory:      /etc/selinux  
Loaded policy name:          targeted  
Current mode:                 enforcing  
Mode from config file:       enforcing  
Policy MLS status:           enabled  
Policy deny_unknown status:  allowed  
Memory protection checking:   actual (secure)  
Max kernel policy version:   33  
[root@localhost ~]#
```

Sécurité sous Linux: SELinux

Contexte SELinux initial des fichiers

- Créez deux fichiers dans le répertoire **/tmp**.
- Les deux fichiers reçoivent le type de contexte **user_tmp_t**

```
[root@host ~]# touch /tmp/file1 /tmp/file2
[root@host ~]# ls -Z /tmp/file*
unconfined_u:object_r:user_tmp_t:s0 /tmp/file1
unconfined_u:object_r:user_tmp_t:s0 /tmp/file2
```

- Dans l'exemple ci-dessous Même étiquette sur le répertoire **/var/www/html** et les fichiers qu'il contient: **httpd_sys_content_t**

```
[root@host ~]# ls -Zd /var/www/html/
system_u:object_r:httpd_sys_content_t:s0 /var/www/html/
[root@host ~]# ls -Z /var/www/html/index.html
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html
```

Sécurité sous Linux: SELinux

Contexte SELinux initial des fichiers

- Déplacez un fichier du répertoire **/tmp** vers le répertoire **/var/www/html**.
- Copiez l'autre fichier dans le même répertoire. Notez l'étiquette obtenue sur chaque fichier.

```
[root@host ~]# mv /tmp/file1 /var/www/html/
[root@host ~]# cp /tmp/file2 /var/www/html/
[root@host ~]# ls -Z /var/www/html/file*
unconfined_u:object_r:user_tmp_t:s0 /var/www/html/file1
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/file2
```

- Le fichier **déplacé a conservé** son étiquette d'origine
- Le fichier **copié a hérité** de l'étiquette du répertoire de destination.

Sécurité sous Linux: SELinux

Contextes par défaut

- **Semanage fcontext -l** liste tous les contextes SELinux par défaut

Exemple:

- La sortie de la commande **semanage** indique que tous les fichiers et sous-répertoires du répertoire **/var/www** ont par défaut le contexte **httpd_sys_content_t**.

```
/var/svn(/.*)?      all files      system_u:object_r:svnserve_content_t:s0
/var/www(/.*)?      all files      system_u:object_r:httpd_sys_content_t:s0
/var/www/dspam(/.*?) all files      system_u:object_r:dspam_content_t:s0
/var/www/gallery/albums(/.*)? all files      system_u:object_r:httpd_sys_rw_content_t:s0
```

Sécurité sous Linux: SELinux

Réétiquetage de système SELinux

- L'exemple suivant crée un répertoire avec un contexte SELinux **default_t**, **hérité** du répertoire parent /.

```
[root@host ~]# mkdir /virtual
[root@host ~]# ls -Zd /virtual
unconfined_u:object_r:default_t:s0 /virtual
```

- La commande **chcon** définit le contexte type du répertoire/virtual sur le type **httpd_sys_content_t**.

```
[root@host ~]# chcon -t httpd_sys_content_t /virtual
[root@host ~]# ls -Zd /virtual
unconfined_u:object_r:httpd_sys_content_t:s0 /virtual
```

- L'exécution de la commande **restorecon** réinitialise le contexte sur la valeur par défaut **default_t**. Remarquez le message **Relabeled**.

```
[root@host ~]# restorecon -v /virtual
Relabeled /virtual from unconfined_u:object_r:httpd_sys_content_t:s0 to unconfined_u:object_r:default_t:s0
[root@host ~]# ls -Zd /virtual
unconfined_u:object_r:default_t:s0 /virtual
```

TP n° 12: Security Enhanced Linux (SELinux)

Atelier 2 : Prise en main de SELinux